# D&D Simulator - LM Studio Application Architecture and Setup Guide

*A source-aligned implementation plan for persistent game state, fast context routing, quest memory, panels, and validated LLM output*

**Project role:** Implementation guide for a local, persistent, single-player text adventure using LM Studio.

**CORE PRINCIPLE**

**The application remembers and validates. The LLM interprets, narrates, and proposes.**

## How this guide fits the existing project

This guide is an implementation companion to the existing **Main instructions** and **Adventure Reference Guide**. It does not replace their gameplay rules. It explains how to turn their concepts into a desktop application that can run for thousands of turns without depending on the LLM to remember the entire campaign.

**Terminology alignment:** Keep the existing names Game-State Ledger, Rolling Story Summary, and Visual Registry in the application. In code, these become database-backed services and records rather than hidden text maintained only inside a chat.

## 1. Target outcome

- **Desktop application.** A single-player text adventure with a narrative pane and persistent profile, inventory, equipment, skills, quests, map, relationships, journal, settings, and visual panels.
- **Local LLM.** LM Studio runs the selected model and exposes a local API to the game application.
- **Persistent campaign.** SQLite stores all authoritative game state, event history, quest summaries, and visual references.
- **Reliable rules.** Deterministic application code performs dice, resource consumption, inventory math, progression, deadlines, and validation.
- **Fast long-term play.** Each turn sends a bounded, relevant snapshot instead of the full campaign transcript.
- **Recoverable saves.** Every approved state change is recorded, and the game can rebuild panels and resume after a restart.

## 2. Recommended architecture

```
Player Input
    -> Desktop UI
    -> Turn Controller
    -> Rules Pre-check + Command Router
    -> Context Builder
         |-> Current scene snapshot
         |-> Relevant character/panel state
         |-> Active quest record
         |-> Recent turn buffer
         |-> Retrieved campaign memories
```

```
      -> LM Studio API (one primary generation call)
      -> Structured Response Parser
      -> Validator + Deterministic Rules Engine
      -> SQLite Transaction
      -> Event Log + Summary Jobs
      -> Panel Refresh + Streamed Narration
```

**Source of truth:** Only committed database state is authoritative. The UI reads from the database, and the LLM receives database snapshots. The LLM must never become the only place where an item, quest state, injury, relationship, or world change exists.

# 3. Suggested desktop technology stack

| Layer | Recommended starting choice | Reason |
|---|---|---|
| Language | Python 3.12+ | Fastest route for LM Studio integration, SQLite, schemas, testing, and desktop prototyping. |
| Desktop UI | PySide6 / Qt | Strong panel layouts, model-view widgets, styling, signals, threading, and desktop packaging. |
| Database | SQLite | Local, transactional, portable, and appropriate for one-player campaign data. |
| Data validation | Pydantic | Validates LLM JSON and application state before database changes. |
| HTTP client | OpenAI Python client or httpx | The OpenAI client can point to LM Studio; httpx provides direct control. |
| Migrations | Alembic or numbered SQL migrations | Allows the schema to evolve without destroying saves. |
| Testing | pytest | Supports unit, integration, replay, and save/load tests. |
| Packaging | PyInstaller initially | Simple first desktop build; revisit packaging after core systems stabilize. |

**Alternative:** C# with Avalonia or WPF is also viable. The architecture in this guide is independent of language; the Python stack is recommended only because it minimizes early integration work.

# 4. LM Studio connection strategy

## 4.1 Start with the OpenAI-compatible API

LM Studio can run a local server from the Developer tab or with `lms server start`. The normal local server address is `http://localhost:1234`. For the first version of the game, use the stateless OpenAI-compatible chat endpoint and let the application own all conversation and game state.

| Option | Use in this project | Recommendation |
|---|---|---|
| POST /v1/chat/completions | Narration plus structured turn result | Best MVP choice because the application explicitly supplies state each turn. |
| POST /v1/responses | Stateful interactions, tools, or later MCP work | Consider after the basic turn pipeline is stable. |
| POST /api/v1/chat | LM Studio native stateful chat and model controls | Useful for advanced management, but not required for the first playable build. |
| GET /v1/models or /api/v1/models | Model discovery and health checks | Use at startup to verify the selected model is available. |

## 4.2 Minimal Python connection example

```
from openai import OpenAI

client = OpenAI(
    base_url="http://localhost:1234/v1",
    api_key="lm-studio",  # Replace with a real token if authentication is enabled.
)

response = client.chat.completions.create(
    model="YOUR_LOADED_MODEL_ID",
    messages=[
        {"role": "system", "content": SYSTEM_INSTRUCTIONS},
        {"role": "user", "content": turn_context_json},
    ],
    temperature=0.65,
    max_tokens=900,
    stream=False,
)

raw_text = response.choices[0].message.content
```

## 4.3 Use structured output for the first implementation

LM Studio supports JSON-schema-constrained structured output on the OpenAI-compatible chat endpoint. Use this to require a predictable response object rather than parsing free-form narration for state changes.

```
TURN_RESULT_SCHEMA = {
    "type": "json_schema",
    "json_schema": {
        "name": "turn_result",
        "strict": True,
        "schema": {
            "type": "object",
            "properties": {
                "narration": {"type": "string"},
                "state_changes": {
                    "type": "array",
                    "items": {"$ref": "#/$defs/state_change"}
                },
                "quest_notes": {
                    "type": "array",
                    "items": {"type": "string"}
                },
                "memory_candidates": {
                    "type": "array",
                    "items": {"type": "string"}
                },
                "suggested_actions": {
                    "type": "array",
                    "items": {"type": "string"},
                    "maxItems": 5
```

```
                }
            },
            "required": [
                "narration", "state_changes", "quest_notes",
                "memory_candidates", "suggested_actions"
            ],
            "additionalProperties": False,
            "$defs": {
                "state_change": {
                    "type": "object",
                    "properties": {
                        "operation": {"type": "string"},
                        "target_type": {"type": "string"},
                        "target_id": {"type": "string"},
                        "field": {"type": ["string", "null"]},
                        "value": {},
                        "reason": {"type": "string"}
                    },
                    "required": [
                        "operation", "target_type", "target_id",
                        "field", "value", "reason"
                    ],
                    "additionalProperties": False
                }
            }
        }
    }
}

response = client.chat.completions.create(
    model=MODEL_ID,
    messages=messages,
    response_format=TURN_RESULT_SCHEMA,
    temperature=0.65,
    max_tokens=900,
)
```

**Model compatibility:** Not every local model follows tools or JSON schemas equally well. Build a small automated evaluation that runs the same 20-50 test turns against candidate models and measures valid JSON rate, rule obedience, continuity, latency, and narration quality.

## 5. Canonical game-state database

The database is the application version of the Game-State Ledger. Store normalized facts, stable IDs, timestamps/turn numbers, and change history. Do not store only formatted panel text.

| Table / aggregate | Purpose | Examples |
|---|---|---|
| campaigns | Campaign identity and global settings | Difficulty, content settings, art style, current turn, game clock. |
| characters | Stable player identity and profile | Name, ancestry, class, appearance lock, level, XP. |
| character_resources | Current and maximum resources | Health, mana, stamina, spell slots, stress. |
| skills / abilities | Progression and unlocks | Skill level, XP, bonus, known spells, feats. |
| items / inventory | Owned item instances and quantities | Condition, weight, value, identification, quest flag. |
| equipment | Slot assignments | Main hand, off hand, armor, trinkets, ammunition. |

| Table / aggregate | Purpose | Examples |
|---|---|---|
| quests / objectives | Active, completed, failed, and rumored quests | Objectives, clues, deadlines, rewards, status. |
| quest_events | Important quest-specific history | Decision, discovery, consequence, participant, location. |
| npcs / relationships | Persistent cast and attitudes | Goals, fears, location, trust, respect, suspicion. |
| locations / location_states | World map and persistent changes | Visited status, routes, structural state, hazards. |
| events | Append-only campaign history | Action, result, entities, importance, tags, before/after references. |
| summaries | Rolling and quest summaries | Current rolling summary, completed quest summaries, campaign arc summary. |
| visual_assets | Visual Registry | Character/location version, state signature, file path, prompt, seed, confirmation. |
| saves | Named checkpoints and recovery metadata | Save label, schema version, current scene, checksum. |

### 5.1 Use stable IDs everywhere

- Use IDs such as char_kaelen, npc_edran, quest_forgotten_shrine, loc_keeper_chamber, and item_keeper_core.
- Narration may use display names, but state changes must target stable IDs.
- Never identify an entity only by its position in a panel or by a model-generated name string.
- Keep a unique constraint where duplicates would break the game, such as one active equipment assignment per slot.

## 6. LLM responsibility boundary

| LLM may propose or produce | Application must decide or validate |
|---|---|
| Narration, descriptions, NPC dialogue, tone, pacing | Whether requested facts are visible to the player. |
| NPC intent and plausible reactions | Relationship bounds and persistence. |
| A requested check, attack, spell, or item use | Legality, resources, targets, range, dice, modifiers, and final result. |
| Quest discoveries and possible consequences | Objective prerequisites, deadlines, rewards, and committed quest status. |
| State-change proposals | Schema validity, ownership, quantities, allowed transitions, and database transaction. |
| Memory candidates and summary language | What is important enough to store and what entities it belongs to. |
| Suggested player actions | The player remains free to type any action. |

**Hard rule:** Never update the database directly from arbitrary generated prose. Every change must pass through a small, enumerated operation type and a validator.

## 7. State-change command design

| Operation family | Examples | Validation |
|---|---|---|
| Resources | damage, heal, spend_resource, restore_resource | Clamp values; verify source and allowed timing. |
| Inventory | add_item, remove_item, consume_item, split_stack | Verify item definition, quantity, capacity, and quest restrictions. |
| Equipment | equip, unequip, damage_equipment | Verify ownership, slot compatibility, and recalculated effects. |
| Progression | award_character_xp, award_skill_xp, unlock_ability | Use application-controlled thresholds and anti-farming rules. |
| Quest | add_quest, update_objective, complete_quest, fail_quest | Verify status transition, prerequisites, deadline, and reward application. |
| World | move_character, advance_time, set_weather, alter_location_state | Verify route, duration, visibility, and persistent consequences. |
| Relationship | adjust_relationship, add_interaction_note | Clamp hidden values and record a reason/evidence event. |
| Visual | request_portrait, request_location_visual, confirm_visual | Check state signature and reuse rules before generation. |

## 8. Turn-by-turn workflow

1. **Capture input.** The UI sends the player text, selected UI action, campaign ID, and current scene ID to the turn controller.
2. **Classify the request.** Recognize menu commands, out-of-character requests, combat actions, dialogue, travel, inventory actions, and ordinary narrative actions.
3. **Perform deterministic pre-checks.** Reject impossible panel actions before calling the LLM when the answer is entirely mechanical.
4. **Build a bounded context package.** Load only current and relevant state, recent turns, active quest details, and a few retrieved memories.
5. **Call LM Studio once.** Request narration and a structured turn result. Stream narration when the selected response format and UI design allow it.
6. **Parse and validate.** Reject malformed output; validate every state change against current database state.
7. **Resolve mechanics.** Run dice and deterministic formulas. If the model requested a roll, the application produces the real result.
8. **Commit atomically.** Apply all approved changes in one SQLite transaction and append the event record.
9. **Refresh derived state.** Update panel views, calculated modifiers, quest display, and active scene snapshot.
10. **Maintain memory.** Add important event records and refresh summaries only when thresholds are reached.

## 9. Context builder: prevent replies from slowing over time

The request sent to the model must be rebuilt each turn and kept inside a fixed budget. Old state remains in SQLite and is retrieved only when relevant. Do not append every prior version of the panels.

| Context section | Normal target | Rule |
|---|---|---|
| Core narrator instructions | 1,500-4,000 tokens | Stable and cached when the runtime supports prompt caching. |
| Current scene snapshot | 500-1,500 | Characters present, visible objects, hazards, exits, time, weather. |
| Relevant player state | 400-1,500 | Only resources, equipment, abilities, or items related to the action. |
| Active quest context | 300-1,200 | Current objectives, discovered clues, deadlines, known consequences. |
| Recent turns | 2,000-6,000 | A rolling window, not the entire transcript. |
| Retrieved memories | 300-1,500 | Usually 3-8 high-relevance event or NPC records. |
| Rolling campaign summary | 600-2,000 | Condensed major continuity only. |
| Reserved response | 600-1,500 | Keep output intentionally bounded by scene importance. |

**Performance target:** A normal turn should use one primary LLM call and a bounded context. Local SQLite reads and validation are normally minor compared with model inference. The prompt should not grow merely because the campaign is older.

### 9.1 Context routing examples

| Player activity | Include | Usually omit |
|---|---|---|
| Combat | Combatants, position, health, conditions, equipped actions, active effects | Completed quests, distant NPCs, full inventory history |
| Merchant | Currency, sellable inventory, merchant stock, faction attitude | Combat log, unrelated location lore |
| Old companion dialogue | NPC profile, relationship state, shared memories, promises, recent dialogue | Every skill and item description |
| Investigation | Current clues, location details, relevant NPC statements, hidden-check request rules | Unrelated world events and distant shops |
| Inventory panel | Current item instances, capacity, equipment links | Narrative transcript and unrelated quest prose |

## 10. Quest memory system

Use each quest as its own bounded memory container. The active quest record is compact enough to include when relevant; the complete quest event log stays in the database.

| Quest record field | Purpose |
|---|---|
| Quest identity and status | Stable ID, title, active/completed/failed/rumored. |
| Known premise | Only information the player has learned. |
| Main and optional objectives | Current status, requirements, and visible progress. |

| Quest record field | Purpose |
|---|---|
| Clues and discoveries | Structured records tied to source events and locations. |
| Important decisions | What the player chose and why it matters. |
| Relevant NPCs and items | Stable entity references, not duplicated biographies. |
| Deadline and clock | Application-controlled time condition. |
| Known rewards and consequences | Do not expose hidden rewards or objectives. |
| Quest narrative summary | Compact LLM-readable continuity. |

### 10.1 During the quest

- Append meaningful quest events, not every sentence of dialogue.
- Update objective status immediately after a validated change.
- Keep the quest summary concise and refresh it after major discoveries, battles, decisions, or approximately every 10 meaningful turns.
- Store exact facts separately from the narrative summary so summary wording cannot overwrite canonical state.

### 10.2 At quest completion or failure

1. Finalize objective states and deadlines.
2. Apply rewards and consequences through deterministic operations.
3. Generate a concise narrative summary from structured quest events.
4. Store permanent world changes, relationship changes, unlocked routes, and unresolved follow-up threads.
5. Move the quest out of the active context set.
6. Add a short quest result to the campaign summary and retain the full event log for search.

## 11. Panels and command mapping

| Existing concept / command | Desktop implementation | LLM involvement |
|---|---|---|
| /profile | Profile panel bound to character and progression views | Optional explanations; no need to regenerate factual values. |
| /inv | Inventory model with categories, filters, quantities, weight, and actions | Descriptions and contextual advice only. |
| /equipment | Equipment slots and derived-stat preview | May describe appearance; code validates changes. |
| /skills | Skill list, XP, bonuses, unlocks | May explain abilities; progression is deterministic. |
| /quests and /quest | Quest list and detail panel | Writes summaries and narrative notes; app controls status. |
| /map | Known-location graph or map panel | Provides descriptions; app controls discovery and routes. |
| /relationships | Observable attitude panel | Produces dialogue/reactions; hidden values remain in state. |
| /journal and /recap | Event timeline plus rolling summary | Summarizes selected records. |

| Existing concept / command | Desktop implementation | LLM involvement |
|---|---|---|
| /portrait and location image | Visual asset viewer/history | Produces image instructions only when visual rules require it. |
| /save, /load, /repair, /resync | Database checkpoint, restore, audit, and correction tools | May explain discrepancies; application performs the operation. |

**Panel efficiency:** Opening a panel should normally require no LLM call. Read the current database state and render it immediately. Call the model only when the player asks for an explanation, narrative interpretation, or action that requires creative resolution.

## 12. Recent memory and long-term retrieval

| Memory layer | Stored form | When included |
|---|---|---|
| Immediate scene | Current scene snapshot | Every narrative turn. |
| Recent turn buffer | Last N player/assistant turns or compact turn records | Every narrative turn, within budget. |
| Active quest | Structured quest record plus short summary | When the scene or action relates to it. |
| NPC memory | Profile, attitude, shared events, promises | When the NPC is present or referenced. |
| Completed quest | Final summary plus consequence records | Only when relevant to current entities or questions. |
| Campaign arc | Rolling summary of major events and unresolved threads | Most narrative turns, kept compact. |
| Full history | Append-only event log | Never sent in full; searched by tags, entities, importance, and text. |

Start retrieval without a vector database. Use entity IDs, quest IDs, location IDs, event type, tags, importance, and recency. Add embeddings later only if ordinary indexed retrieval cannot find sufficiently relevant memories.

## 13. Rolling Story Summary

- **Purpose.** Provide the LLM with compressed narrative continuity without replacing canonical records.
- **Content.** Recent major events, discoveries, important NPC interactions, meaningful decisions, current conflicts, unresolved mysteries, and immediate objectives.
- **Refresh triggers.** Approximately every 10 meaningful turns, after a major battle, quest result, region change, relationship change, world-state change, or save.
- **Versioning.** Keep the newest summary active and preserve older summary versions for debugging and save recovery.
- **Safety.** Generate the summary from committed events. Never apply database state changes merely because a summary says something happened.

## 14. Visual Registry implementation

Map the existing visual continuity rules to persistent asset records. Store files on disk and metadata in SQLite.

| Field | Example purpose |
|---|---|
| asset_id and entity_id | Links a file to a character, location, item, or scene. |
| asset_type and version | Canonical portrait, active portrait, location state, history version. |
| state signature | Structural condition, event state, time, season, weather, viewpoint, equipment set. |
| file path / URI | Exact reusable asset location. |
| prompt and negative prompt | Reproduction and continuity aid. |
| reference asset IDs | Canonical character or prior confirmed versions. |
| seed / model / workflow | Store when available. |
| confirmation state | Pending, confirmed, active, superseded, rejected. |

**Reuse rule:** When a valid stored location state exists, load the exact asset. Do not ask the LLM or image system to make a similar replacement. Generate a new version only for a qualifying drastic state change or when no valid asset exists.

## 15. Save, load, repair, and replay

- Use normal SQLite persistence continuously; a save is a named checkpoint, not the first time state is written.
- Store a schema version and migration history with every campaign.
- Before a named save, commit pending changes, refresh the rolling summary, and record active scene and visual references.
- For load, open a database transaction, verify schema compatibility and referenced assets, then rebuild all panels from state.
- For repair, show the discrepancy, evidence events, proposed correction, and resulting panel changes before committing when user confirmation is appropriate.
- Create a replay test that re-applies event operations to a known checkpoint and compares the resulting state checksum.

## 16. Failure handling

| Failure | Required behavior |
|---|---|
| LM Studio not running | Show a clear connection status, preserve player input, and offer retry after the server is available. |
| Model not loaded | List available models or instruct the user to load the configured model; do not corrupt the turn. |
| Timeout or interrupted generation | Cancel cleanly; commit no model-proposed state changes. |
| Malformed JSON | Retry once with a repair request or reject the result; never guess missing state operations. |
| Invalid state operation | Reject that operation, log the reason, and either continue with valid independent operations or retry the turn atomically. |
| Database failure | Roll back the entire turn and retain the previous committed state. |
| Missing visual asset | Mark the visual pending and continue text play; never claim it was restored. |

| Failure | Required behavior |
|---|---|
| Context budget exceeded | Trim retrieved memories and old recent turns, then use the summary; never silently remove current scene essentials. |

## 17. Security and privacy

- Bind LM Studio to localhost unless network access is intentionally required.
- If the server is exposed beyond localhost, enable authentication and restrict firewall access.
- Store API tokens outside source control and redact them from logs.
- Never expose a generic database-write tool to the model. Expose narrow operations with explicit schemas and permissions.
- Treat imported campaign files, mods, and prompts as untrusted input; validate paths and prevent traversal outside the campaign asset directory.
- Keep backups before database migrations and before destructive repair operations.

## 18. Performance plan

| Technique | Effect |
|---|---|
| One primary LLM call per narrative turn | Avoids cumulative sequential latency. |
| No LLM call for ordinary panel opens | Panels feel immediate. |
| Fixed context budget | Latency and memory usage remain relatively stable across long campaigns. |
| Current snapshots, not historical panel copies | Prevents prompt growth. |
| Delta state output | The model returns only proposed changes, not every unchanged record. |
| Indexed local retrieval | Finds relevant history without loading full logs. |
| Streaming narration | Improves perceived responsiveness even when total generation time is unchanged. |
| Summary work outside the critical path | Quest and campaign summaries can run after the turn commit or at checkpoints. |
| Small output caps by activity | Routine actions remain concise; major scenes may use larger budgets. |
| Model loaded at practical context length | Avoids spending VRAM on an advertised maximum that the game rarely uses. |

## 19. Testing requirements

| Test group | Minimum coverage |
|---|---|
| Schema tests | Valid/invalid turn results, unknown operations, missing fields, wrong entity IDs. |
| Rules tests | Dice, modifiers, damage, healing, resource costs, inventory, equipment, XP, quest transitions. |
| Database tests | Atomic commit, rollback, migrations, save/load, duplicate prevention. |

| Test group | Minimum coverage |
|---|---|
| Context tests | Correct routing for combat, merchant, dialogue, investigation, travel, and menus. |
| Memory tests | Relevant event retrieval, unrelated-event exclusion, summary refresh triggers. |
| LLM evaluations | JSON success rate, hallucinated inventory, hidden-information leakage, continuity, latency, narration quality. |
| Replay tests | Known sequence produces the expected final checksum and visible panel state. |
| Failure tests | Server offline, timeout, malformed response, invalid change, database lock, missing visual asset. |

## 20. Recommended implementation phases

| Phase | Deliverable | Do not add yet |
|---|---|---|
| 1. Skeleton | Desktop window, navigation, campaign folder, settings, logging | Advanced styling, images, mods |
| 2. State | SQLite schema, repositories, migrations, seed campaign | LLM integration |
| 3. Panels | Profile, inventory, equipment, skills, quests read from database | Narrative generation |
| 4. LM connection | Health check, model selection, single prompt/response test | Tool autonomy |
| 5. Structured turns | Context package, JSON schema, parser, validator, narration display | Complex combat |
| 6. Core gameplay | Movement, dialogue, checks, inventory actions, quest updates | Visual generation |
| 7. Memory | Event log, recent buffer, quest summaries, rolling summary, retrieval | Embeddings unless needed |
| 8. Combat/rules | Initiative, actions, resources, injuries, rewards, tests | Unbounded custom rules |
| 9. Visual Registry | Asset storage, state signatures, reuse/history/confirmation | Automatic generation until registry is reliable |
| 10. Reliability | Save/load, repair, replay, backups, packaging, user testing | Feature expansion before stability |

## 21. Definition of a playable minimum version

- The player can create or load one character and campaign.
- Profile, inventory, equipment, skills, and quest panels read directly from SQLite.
- The app connects to a configured LM Studio model and shows a connection status.
- One player action produces narration plus schema-valid proposed changes.
- The validator prevents nonexistent items, negative currency, illegal equipment, and invalid quest transitions.
- A complete turn commits atomically and refreshes the affected panels.
- Recent turns, active quest state, and a rolling summary are included within a fixed context budget.

- The campaign survives application restart and loads to the exact prior committed state.
- Automated tests cover the turn parser, major validators, and save/load path.

## 22. Instructions for ChatGPT while building the application

**Use this section in the new project:** Copy the following block into the ChatGPT project instructions together with the existing Main instructions, Adventure Reference Guide, and the separate Project Workflow document.

```
You are helping build a desktop D&D-style single-player adventure application that connects to a local LLM
through LM Studio.

Authoritative project documents:
1. Main instructions - gameplay behavior and player-facing rules.
2. Adventure Reference Guide - detailed state, menu, progression, quest, combat, continuity, and visual
rules.
3. LM Studio Application Architecture and Setup Guide - technical architecture and implementation
boundaries.
4. Simple Project Workflow - ordered implementation steps.

Working rules:
- Treat the application database as the source of truth.
- The LLM narrates and proposes structured changes; deterministic code validates and commits them.
- Preserve the existing terminology: Game-State Ledger, Rolling Story Summary, and Visual Registry.
- Work on only the current workflow step unless I explicitly authorize more.
- Do not rewrite unrelated files or redesign completed systems without a stated reason.
- Prefer small, testable modules and complete code that can run with minimal modification.
- Include input validation, error handling, database transactions, logging, and tests where relevant.
- Keep normal turns to one primary LLM call and a bounded context package.
- Do not append the entire campaign history or every panel on each request.
- Do not allow arbitrary model-generated database writes.
- End each work response with: files changed, tests run, unresolved issues, and the single next step.
- Stop after completing the current step and ask whether I want to continue.
```

## Official LM Studio references

Verified against official LM Studio developer documentation. These links are included because API capabilities and endpoints may evolve.

- Local API server overview
- LM Studio REST API overview
- OpenAI-compatible endpoints
- Structured JSON output
- Tool use / function calling
- Server settings and authentication
- Model loading and context settings

# IMPLEMENTED APPLICATION STATUS - RELEASE 24

**Last reconciled: 2026-07-23**

The original architecture remains the design foundation. This section records the as-built Release 24 state and supersedes implementation-stage assumptions. See IMPLEMENTATION_HISTORY.md for the release-by-release ledger.

## As-built platform baseline

- Python and PySide6 desktop application, packaged for Windows as a one-folder PyInstaller distribution.
- Application version 0.25.0 with SQLite schema 17 and forward migrations that preserve historical campaigns.
- Typed repositories and services own persistence, validation, derived state, and transaction boundaries; widgets remain projections and command sources.

## LM Studio integration

- Settings discovers installed and loaded models, permits explicit loading through LM Studio's management API, and persists only a confirmed loaded selection.
- Narrative turns, NPC dialogue, creation randomization, and campaign generation use bounded schemas, compatibility repair, cancellation, timeout, and deterministic fallback behavior.
- The three-sample evaluator records schema reliability, retry behavior, latency, token rate, context, memory metadata, and bounded narration quality without granting the model game-state authority.
- The shared narrative normalizer removes transport backticks and otherwise preserves the model-authored text and declared semantic kind without heuristic splitting or quote insertion.

## Implemented state domains

- Core ledger, character creation, inventory/equipment, structured turns, combat, abilities/spells, enemy archetypes, rewards, quest lifecycle, map/time/travel/camp, NPC relationships, Visual Registry, reliability, commerce, and crafting.
- Immediate commits provide autosave semantics. Checkpoints, recovery backups, integrity inspection, repair reports, and replay/checksums provide explicit recovery and audit paths.

## Deterministic visual resolution

- Release 25 packages the 1,686-record semantic manifest and numbered SVG files. Resolution prefers exact semantic identity before rarity-aware stable matching; no per-item model or image-generation call is required.
- Icons appear only in the Default theme and refresh during live theme switching. Existing High Fantasy and Modern Dark theme files remain independent.
- Inventory and equipment cards use left-positioned 42-pixel SVG icons while retaining compact dimensions. Hover cards render a 64-pixel icon and exist only for the active hover.

## Packaging and verification

- Source and packaged smoke paths verify startup, writable configuration, schema readiness, release-specific behavior, and clean shutdown.
- The packaged application includes themes, fonts, the semantic manifest and all numbered SVGs, UI icons, and runtime dependencies. Campaign data remains in the user's writable application-data location.

## Release 25 semantic catalog and trade bridge

A cached application catalog loads all 1,686 semantic records with stable IDs, categories, subtypes, tags, tiers, and rarity mapping. Tier weights are 100, 35, 12, 4, and 1.

Combat, quests, merchant initialization, and non-combat discovery call the same deterministic selector. Existing inventory and reward tables remain authoritative, so schema 17 does not change.

The non-combat action context carries eight scene candidates and exact allowed IDs. Discovery is accepted only after an explicit search, open, loot, investigate, or explore action.

Adventure commerce intent reuses the existing People dialogue and Commerce services. NPC dialogue commits player, monologue, and spoken entries into narrative_messages for Return-to-Adventure continuity.